

Chapter 4

PoS and Hidden Markov Models

Er. Shiva Ram Dam

Overview: 4 hrs

1. PoS Tagsets
2. Rule-based PoS tagging
3. Stochastic PoS tagging
4. Transformation based tagging

English Parts of Speech

- **Noun (person, place or thing)**
 - Singular (NN): dog, fork
 - Plural (NNS): dogs, forks
 - Proper (NNP, NNPS): John, Springfields
 - Personal pronoun (PRP): I, you, he, she, it
 - Wh-pronoun (WP): who, what
- **Verb (actions and processes)**
 - Base, infinitive (VB): eat
 - Past tense (VBD): ate
 - Gerund (VBG): eating
 - Past participle (VBN): eaten
 - Non 3rd person singular present tense (VBP): eat
 - 3rd person singular present tense: (VBZ): eats
 - Modal (MD): should, can
 - To (TO): to (to eat)
- **Adjective (modify nouns)**
 - Basic (JJ): red, tall
 - Comparative (JJR): redder, taller
 - Superlative (JJS): reddest, tallest
- **Adverb (modify verbs)**
 - Basic (RB): quickly
 - Comparative (RBR): quicker
 - Superlative (RBS): quickest
- **Preposition (IN):** on, in, by, to, with
- **Determiner:**
 - Basic (DT) a, an, the
 - WH-determiner (WDT): which, that
- **Coordinating Conjunction (CC):** and, but, or,
- **Particle (RP):** off (took off), up (put up)

4.1 POS tagging in NLP

- POS tagging means **telling the computer what each word is** in a sentence.

- Is it a **noun**?

Is it a **verb**?

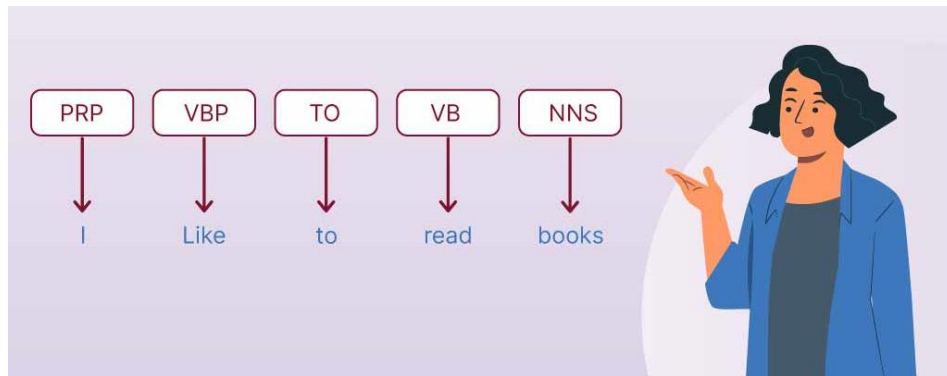
Is it an **adjective**?

Is it an **adverb**?

- Example:

Sentence: “Ram eats rice daily.”

- Ram → noun
- eats → verb
- rice → noun
- daily → adverb



- So POS tagging gives **labels** to each word.

Can you help me with the can?

- Parts of speech(PoS) tagging is crucial for **syntactic** and **semantic analysis**.
- Therefore, for something like the sentence above, the word “can” has several semantic meanings.
- The first “can” is used for question formation.
- The second “can” at the end of the sentence is used to represent a container.
- The first “**can**” is a **verb**, and the second “**can**” is a **noun**.
- Giving the word a specific meaning allows the program to handle it correctly in both semantic and syntactic analysis.

Why do we do POS Tagging?

- Because computers don't understand grammar like humans.
 - POS tagging helps computers:
 - understand **actions** (verbs)
 - identify **things/people** (nouns)
 - find **describing words** (adjectives)
 - understand **how/when** (adverbs)

Analogy: POS Tagging Like Labeling Things in a Kitchen

- Imagine your kitchen has many items: spoon, fork, pan, stove. To cook properly, you must **know what each item is used for**.
- Similarly:
 - Words = items in kitchen
 - POS tags = labels (noun, verb, adjective)
- If a machine knows:
 - which word is an action → verb
 - which word is a thing → noun
- ...it can “cook” correct meaning!

Example:

- For the sentence "***The quick brown fox jumps over the lazy dog***":
- POS tags are:

The: Determiner (DT)

quick: Adjective (JJ)

brown: Adjective (JJ)

fox: Noun (NN)

jumps: Verb (VBZ)

over: Preposition (IN)

the: Determiner (DT)

lazy: Adjective (JJ)

dog: Noun (NN)

Key Concepts in POS Tagging

- **Parts of Speech:** These are categories like nouns, verbs, adjectives, adverbs, etc that define the role of a word in a sentence.
- **Tagging:** The process of assigning a specific part-of-speech label to each word in a sentence.
- **Corpus:** A large collection of text data used to train POS taggers.

Working of POS Tagging

1. **Tokenization:** The input text is split into individual tokens (words or subwords), this step is necessary for further analysis.
2. **Preprocessing:** The text is cleaned such as converting it to lowercase and removing special characters, to improve accuracy.
3. **Loading a Language Model:** Tools like [NLTK](#) or [SpaCy](#) use pre-trained language models to understand the grammatical rules of the language, these models have been trained on large datasets.
4. **Linguistic Analysis:** The structure of the sentence is analyzed to understand the role of each word in context.
5. **POS Tagging:** Each word is assigned a part-of-speech label based on its role in the sentence and the context provided by surrounding words.
6. **Evaluation:** The results are checked for accuracy. If there are any errors or misclassifications, they are corrected.

4.2 PoS Tagsets

- A **PoS tagset** is a collection of tags used to classify words.
- Different tagsets have different numbers of tags.
- **Why Do We Need POS Tagsets?**
 - To **standardize** how words are tagged
 - So all NLP tools follow the **same labels**
 - For research, machine learning, translation, parsing, etc.
- Three Common POS Tagsets
 1. Universal POS tagset
 2. Brown Corpus Tagset
 3. Penn Treebank Tagset

1. Universal POS tagset

- 17 tags
- Simple
- Works for many languages
- Easy for students
- **Why popular?**
Simple, minimal, used in spaCy and many NLP tools.

Tag	Meaning	Example
NOUN	Noun	dog, Nepal
VERB	Verb	run, eat
ADJ	Adjective	big, tall
ADV	Adverb	slowly, quietly
PRON	Pronoun	he, they
DET	Determiner	a, the
ADP	Adposition (prepositions)	in, on
CONJ	Conjunction	and, but
NUM	Number	one, 100
PART	Particle	not, 's
PRT	Particle (older name)	up (as in “pick up”)
PROPN	Proper Noun	Ram, Pokhara
INTJ	Interjection	oh!, wow!
AUX	Auxiliary Verb	is, can
SCONJ	Subordinating Conjunction	although, because
PUNCT	Punctuation	. , ;
X	Other	foreign words

2. Penn Treebank tagset

- Penn Treebank tagset (45 tags)
- This is one of the most widely used tagset in NLP.
- Used in research
- This tagset gives specific forms of verbs, noun, etc.

Nouns

Tag	Meaning	Example
NN	Noun (singular)	book
NNS	Noun (plural)	books
NNP	Proper noun (singular)	Nepal
NNPS	Proper noun (plural)	Himalayas

Verbs

Tag	Meaning	Example
VB	Verb base form	eat
VBD	Verb past tense	ate
VBG	Verb gerund/continuous	eating
VBN	Verb past participle	eaten
VBP	Verb non-3rd person	run
VBZ	Verb 3rd person singular	runs

Adjectives

Tag	Meaning	Example
JJ	Adjective	big
JJR	Comparative	bigger
JJS	Superlative	biggest

Pronouns

Tag	Example
PRP	he, she
PRP\$	his, her

Adverbs

Tag	Meaning	Example
RB	Adverb	quickly
RBR	Comparative	faster
RBS	Superlative	fastest

Determiners

- DT** – the, a
- PDT** – all, both
- WDT** – which, what

Conjunctions

- CC** – and, but
- IN** – in, into (preposition), because (subordinating conj.)

Others

- CD** – Numbers
- EX** – Existential “there”
- FW** – Foreign words
- UH** – Interjections
- SYM** – Symbols
- LS** – List item marker (1., 2.)
- TO** – “to”
- MD** – Modal verbs (can, will, should)

3. Brown Corpus tagset

- Original **Brown corpus** used a large set of **87 POS tags**.
- It is the most detailed tagset used in corpus linguistics
- Used in early NLP research

- **.** (sentence terminator): ., !, ?
- **(** (left paren): ([{
- **)** (right paren):)] }
- **,** (comma): ,
- **:** (colon): ;
- **ABL** (pre-qualifier): quite, very
- **ABN** (pre-quantifier): half, all
- **ABX** (pre-quantifier): both
- **AP** (post-determiner): next, old
- **AT** (article): the, a
- **BE** (be, present): am, is, are
- **BED** (were): were
- **BEDZ** (was): was
- **BEG** (being): being

- **BEM** (am): am
- **BEN** (been): been
- **BER** (are): are, art
- **DO** (do): do
- **DOD** (did): did
- **DOZ** (does): does
- **HV** (have): have
- **HVD** (had): had (past tense)
- **HVG** (having): having
- **HVN** (have, past participle): had
- **MD** (modal): can, could, will, would
- **NP** (proper noun, singular): John, city, Red
- **NPS** (proper noun, plural): Americans
- **NR** (adverbial noun): home, today, west
- **RB** (adverb): fast, quickly

Comparision

Tagset	Tags	Difficulty	Usage
Universal	17	Easy	Modern NLP tools
Penn Treebank	45	Medium	Research, NLTK, CoreNLP
Brown	87	Hard	Historical NLP

Example 1

```
import nltk
# Download required resources (only once)
nltk.download('punkt_tab')          # For tokenizing
nltk.download('averaged_perceptron_tagger_eng') # For POS tagging

# Input sentence
sentence = "The quick brown fox jumps over the lazy dog."

# Tokenize the sentence
tokens = nltk.word_tokenize(sentence)

# Perform POS tagging
pos_tags = nltk.pos_tag(tokens)

# Display results
print("POS Tags:")
for word, tag in pos_tags:
    print(f"{word} --> {tag}")
```

```
POS Tags:
The --> DT
quick --> JJ
brown --> NN
fox --> NN
jumps --> VBZ
over --> IN
the --> DT
lazy --> JJ
dog --> NN
```

Example 2

```
Import nltk
```

```
#POS tagging
```

```
tag=nltk.pos_tag(["Studying", "Study"])  
print(tag)
```

```
[('Studying', 'VBG'), ('Study', 'NN')]
```

Example 3

#PoS tagging example :

```
sentence = "A very beautiful young lady is walking on the beach"
```

#Tokenizing words :

```
tokenized_words = word_tokenize(sentence)
```

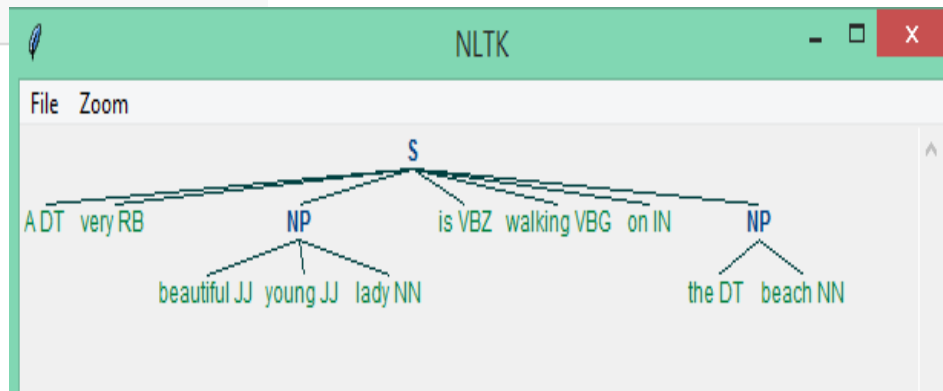
```
for words in tokenized_words:
```

```
    tagged_words = nltk.pos_tag(tokenized_words)
```

```
tagged_words
```

```
[('A', 'DT'),  
 ('very', 'RB'),  
 ('beautiful', 'JJ'),  
 ('young', 'JJ'),  
 ('lady', 'NN'),  
 ('is', 'VBZ'),  
 ('walking', 'VBG'),  
 ('on', 'IN'),  
 ('the', 'DT'),  
 ('beach', 'NN')]
```

- In this example, we can see that we have successfully extracted the **noun phrase** from the text.



4.3 Types of PoS Tagging

- a) Rule Based POS Tagging
- b) Statistical POS Tagging
- c) Transformation-based tagging (TBT)
- d) Hidden Markov Model POS tagging

a) Rule-based PoS tagging

- Rule-based part-of-speech (POS) tagging is a method of labeling words with their corresponding parts of speech **using a set of pre-defined rules.**
- In rule-based POS tagging system, words are assigned POS tags **based on their characteristics and the context in which they appear.**
- **For example,** a rule-based POS tagger might assign the **tag “noun”** to **any word that ends in “-tion” or “-ment,”** as these suffixes are often used to form **nouns.**
- However, they can be less accurate and less efficient than machine learning-based taggers, especially for tasks with large or complex datasets.
- **Advantages:**
 - Transparent and explainable.
 - Does not require annotated data.
- **Disadvantages:**
 - Poor coverage.
 - Cannot handle ambiguity well.

- Here is an example of how a rule-based POS tagger might work:
 - Define a set of rules for assigning POS tags to words.
- For example:
 - If the word **ends in “-tion,”** assign the tag **“noun.”**
 - “Nation” would be tagged as “noun” based on this first rule.
 - If the word **ends in “-ment,”** assign the tag **“noun.”**
 - “Investment” would be tagged as “noun” based on this second rule.
 - If the **word is all uppercase,** assign the tag **“proper noun.”**
 - “UNITED” would be tagged as “proper noun” based on this third rule.
 - If the word is a **verb ending in “-ing,”** assign the tag **“verb.”**
 - “Running” would be tagged as “verb” based on this fourth rule.

Example 4:

```
def simple_rule_based_pos_tagger(tokens):
    tagged = []
    for word in tokens:
        word_lower = word.lower()
        if word_lower in ['a', 'an', 'the']:
            tag = 'DET' # Determiner
        elif word_lower in ['is', 'am', 'are', 'was', 'were']:
            tag = 'VERB'
        elif word_lower.endswith('ing'):
            tag = 'VERB'
        elif word_lower in ['book', 'dog', 'library']:
            tag = 'NOUN'
        elif word_lower in ['quickly', 'slowly']:
            tag = 'ADV'
        elif word_lower in ['quick', 'lazy']:
            tag = 'ADJ'
        else:
            tag = 'UNK' # Unknown
        tagged.append((word, tag))
    return tagged
```

```
# Example sentence
sentence = "The dog is running quickly to the library"

# Tokenize (split by spaces here for simplicity)
tokens = sentence.split()

# Tag the sentence
tagged_sentence = simple_rule_based_pos_tagger(tokens)

# Display result
for word, tag in tagged_sentence:
    print(f"{word} --> {tag}")
```

```
The --> DET
dog --> NOUN
is --> VERB
running --> VERB
quickly --> ADV
to --> UNK
the --> DET
library --> NOUN
```

b) Stochastic PoS tagging

- It **uses probabilistic models** to assign grammatical categories (e.g noun, verb, adjective) to words in a text.
- Unlike rule-based methods which rely on handcrafted rules, **it learns patterns from large annotated corpora** using machine learning techniques.
- These models **calculate the likelihood of a tag based on a word** and its context, helping to resolve ambiguities and handle complex grammar.
- Common models include:
 - **Hidden Markov Models (HMMs)**
 - **Conditional Random Fields (CRFs)**

- **Characteristics:**
 - Based on probabilities.
 - Requires a training dataset.
 - Doesn't handle unknown words well.
 - Often uses different datasets for training and testing.
- **Advantages:**
 - More robust than rules.
 - Can capture tag sequence patterns.
- **Disadvantages:**
 - Needs large annotated corpora.
 - Performance drops with out-of-vocabulary words.

Example 5:

```
import nltk

# Download required data
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

# Input sentence
sentence = "She enjoys reading books in the quiet library."

# Tokenize sentence
tokens = nltk.word_tokenize(sentence)

# Stochastic POS tagging using pre-trained model
pos_tags = nltk.pos_tag(tokens)

# Display result
print("Stochastic POS Tags:")
for word, tag in pos_tags:
    print(f"{word} --> {tag}")
```

Stochastic POS Tags:

```
She --> PRP
enjoys --> VBZ
reading --> VBG
books --> NNS
in --> IN
the --> DT
quiet --> JJ
library --> NN
```

c) Transformation based tagging

- Aka Brill Tagging
- Hybrid of rule-based and statistical methods.
- Transformation-based tagging uses a series of rules to transform the tags of words in a text.
- In TBT, **a set of rules is defined** to transform the tags of words in a **text based on the context in which they appear**.
- For example,
 - a rule might change the tag of a **verb to a noun** if it appears after a determiner such as “the.” **The work is fine.** Here, **work** is **noun**
- Characteristics:
 - Combines rule-based and stochastic approaches.
 - Rules are learned from data automatically.
 - Easy to understand and debug.
 - Faster than some statistical models but takes time to train.

How it works:

- **Initial Guess:** Starts with an initial tagging, often using a simple method like word frequency.
- **Transformation Rules:** Applies rules to improve the tagging. For example, if "play" was initially tagged as a noun but follows "to," a rule might change it to a verb.

d) Hidden Markov Model POS tagging

- HMM = **Hidden Markov Model**
- It is a **statistical method** to guess the correct POS tag (noun, verb, adjective...) for each word.
- **Why “hidden”?**
Because the POS tags are not directly visible — they are *hidden*.
We only see words (visible), and we try to guess the tags (hidden).
- HMM POS tagging is a smart guessing method.
- It guesses POS tags by looking at:
 - which tags come together
 - which tag fits the word
- It chooses the **most likely tag sequence**.

Simple Idea

- HMM POS Tagging tries to answer two questions:
 - **A. What POS tag usually comes before this tag?**
 - Example:
 - Determiner (the) → often followed by a noun
 - Pronoun (he/she) → often followed by a verb
 - **B. What POS tag usually belongs to this word?**
 - Example:
 - “eat” → usually a verb
 - “quickly” → usually an adverb
 - So HMM uses:
 - **Tag-to-Tag probability** (What tag comes next?)
 - **Word-to-Tag probability** (Which tag fits the word?)
 - Then it chooses the **most likely sequence** of POS tags.

Simple English Example

- Sentence:
"They can fish."
- Two meanings:
 - "can" → modal verb
 - "fish" → verb OR noun
- HMM prediction:
 - After "They" → likely a verb
 - "can" → modal verb
 - After modal verb → main verb
So "fish" becomes **verb**
- Final tags:
They/PRP can/MD fish/VB

HMM use case1

Given the sentence, disambiguate the appropriate POS tag for “park” using HMM model



Emission Probability

	I	drove	to	park
VB	0	0.1	0	0.68
TO	0	0	0.99	0
NN	0	0.85	0	0.8
PPSS	0.37	0	0	0

Transition Probability

	VB	TO	NN	PPSS
<s>	0.19	0.43	0.41	0.67
VB	0.38	0.35	0.47	0.70
TO	0.47	0	0.83	0
NN	0.40	0.16	0.87	0.45
PPSS	0.23	0.79	0.12	0.14

$$P(NN|TO) = 0.83$$

$$P(VB|TO) = 0.47$$

$$P(\text{park} | NN) = 0.8$$

$$P(\text{park} | VB) = 0.68$$

$$P(NN|TO) \times P(\text{park}|NN) = 0.83 \times 0.8 = 0.664$$

$$P(VB|TO) \times P(\text{park}|VB) = 0.47 \times 0.68 = 0.3196$$



Source:

<https://www.youtube.com/watch?v=j5dsNym8t1g>

To compute $P(\text{Park} \mid \text{NN})$:

- Suppose in the corpus, "Park" appears 80 times as a noun, and the total number of noun-tagged words is 100.

$$P(\text{Park} \mid \text{NN}) = \frac{80}{100} = 0.8$$

Similarly, if "Park" appears 20 times as a verb out of 1000 total verbs:

$$P(\text{Park} \mid \text{VB}) = \frac{20}{1000} = 0.02$$

So emission probabilities are calculated from observed frequencies.

To calculate $P(\text{NN} \mid \text{TO})$:

- If "TO" is followed by "NN" in 800 sentences out of 1000 where "TO" occurs:

$$P(\text{NN} \mid \text{TO}) = \frac{800}{1000} = 0.8$$

Similarly, if "TO" is followed by "VB" in 200 cases:

$$P(\text{VB} \mid \text{TO}) = \frac{200}{1000} = 0.2$$

4.4 Steps involved in POS tagging

Step	Description
1. Tokenization	Split sentence into words Example: "She enjoys playing tennis." → ["She", "enjoys", "playing", "tennis", "."]
2. Lexical lookup	Get possible tags for each word Example: "play" → [VB, NN], "she" → [PRP]
3. Disambiguation	Use rules/statistics/ML to choose correct tag Example rule: "If the word after 'to' is a verb, tag it as VB."
4. Tagging	Assign final tag to each word Example: "She/PRP enjoys/VBZ playing/VBG tennis/NN ./."
5. Post-processing	Handle errors or unknown words

4.5 Use of POS tagging in NLP

1. **To understand the grammatical structure of a sentence:** By labeling each word with its POS, we can better understand the syntax and structure of a sentence. This is useful for tasks such as machine translation and information extraction, where it is important to know how words relate to each other in the sentence.
2. **To disambiguate words with multiple meanings:** Some words, such as “bank,” can have multiple meanings depending on the context in which they are used. By labeling each word with its POS, we can disambiguate these words and better understand their intended meaning.
3. **To improve the accuracy of NLP tasks:** POS tagging can help improve the performance of various NLP tasks, such as named entity recognition and text classification. By providing additional context and information about the words in a text, we can build more accurate and sophisticated algorithms.
4. **To facilitate research in linguistics:** POS tagging can also be used to study the patterns and characteristics of language use and to gain insights into the structure and function of different parts of speech.

4.6 Challenges in POS tagging in NLP

1. **Ambiguity:** Some words can have multiple POS tags depending on the context in which they appear, making it difficult to determine their correct tag. For example, the word “bass” can be a noun (a type of fish) or an adjective (having a low frequency or pitch).
2. **Out-of-vocabulary (OOV) words:** Words that are not present in the training data of a POS tagger can be difficult to tag accurately, especially if they are rare or specific to a particular domain.
3. **Complex grammatical structures:** Languages with complex grammatical structures, such as languages with many inflections or free word order, can be more challenging to tag accurately.
4. **Lack of annotated training data:** Some languages or domains may have limited annotated training data, making it difficult to train a high-performing POS tagger.
5. **Inconsistencies in annotated data:** Annotated data can sometimes contain errors or inconsistencies, which can negatively impact the performance of a POS tagger.

End of chapter 4